DEEP REINFORCEMENT LEARNING • NETWORK PROTOCOL OPTIMIZATION

TCP-RL-ns3: Reinforcement Learning-Driven TCP Congestion Control in ns-3

Autonomous Bandwidth-Delay Product (BDP) Tracking & Low-Latency Queue Management under Non-Stationary Bottlenecks

Authors: Shubh Shetgaonkar & Abinav Pallathoor	**Simulator:** ns-3.40 + ns3-gym (ZeroMQ / Protobuf)
Framework: PyTorch & Stable-Baselines3 (PPO)	**Repository:** github.com/shetgaonkarshubh/tcp-rl-ns3
Control Domain: BBR-style Congestion Window Adjustment	**License:** MIT Open Source License

BACKGROUND & CHALLENGES

The Congestion Control Dilemma: Bufferbloat vs. Underutilization

Traditional heuristics struggle with modern non-stationary and latency-sensitive networks.

Limitations of Legacy TCP

1. Loss-Based Protocols (Reno, Cubic):

- Assume packet drops are the primary signal of congestion.
- Continuously push packets into intermediate router buffers until tail-drop occurs, inducing massive **bufferbloat** and latency inflation.
- Halve cwnd aggressively upon loss, severely degrading throughput on modern cellular/Wi-Fi links with stochastic wireless drops.

2. Delay-Based & Model-Based Approaches (BBR, Vegas):

- Aim to operate at the Kleinrock optimal operating point (maximum throughput, minimum RTT).
- Hand-crafted state machines can struggle when bottleneck bandwidth fluctuates unpredictably or when competing with aggressive flows.

The Deep RL Opportunity

Why Deep Reinforcement Learning (DRL)?

- **Non-Linear State Perception:** Directly processes multi-dimensional telemetry (RTT trends, flight sizes, queue inflation) instead of simple threshold logic.
- **Adaptive Control Policies:** Learns asymmetric adjustments—rapidly draining queues on bottleneck constrictions while gently probing during expansions.

Project Objectives:

- Couple **ns-3.40** with **Stable-Baselines3** via high-speed IPC (ns3-gym).
- Formulate a BBR-style observation and reward schema that targets the dynamic BDP sweet-spot.
- Train a resilient PPO agent robust to severe, sudden bandwidth shifts.

SYSTEM ARCHITECTURE

Co-Simulation Pipeline: ns-3 Simulator & Python RL Agent

A bi-directional, synchronous IPC architecture bridging C++ discrete-event simulation with PyTorch.

Simulation (C++17)	**ns-3.40 Network Core** rl-tcp.cc	• Simulates realistic point-to-point network topologies with bottleneck links. • Hooks into TcpSocketState to intercept cwnd, ssThresh, RTT, segments in flight, and congestion states (CA_OPEN, CA_RECOVERY, etc.). • Executes commanded cwnd and ssThresh updates at each 100 ms control interval.
IPC Bridge	**ns3-gym Module** ZeroMQ / Protobuf	• Synchronous IPC socket over localhost port 7144. • Serializes raw C++ telemetry into Protocol Buffer messages and returns agent decisions back to the simulator with zero disk overhead.
Gymnasium Environment	**CustomTcpEnv** tcp_env.py	• Emulates dynamic network environments with a stochastic Markov bottleneck engine. • Calibrates physical Base RTT (84 ms) and derives time-invariant 8-dim observations. • Computes multi-objective reward balancing BDP tracking, throughput, and delay penalty.
Learning Agent	**PPO / PyTorch** train.py	• Stable-Baselines3 PPO with customized MLP policy (SiLU activations). • Maintains running reward normalization via VecNormalize.

ENVIRONMENT MODELING

Stochastic Bottleneck Markov Engine & Queue Dynamics

Realistic non-stationary evaluation forcing the agent to track unpredictable capacity jumps.

Markov Bottleneck Engine

1. Random Capacity Transitions:

- Real-world links (5G, satellite, multi-tenant WAN) feature abrupt bandwidth variations.
- The environment randomly samples target BDP levels uniformly in **[15.0, 36.5] MSS**.
- Enforces severe shifts: **|Δ BDP|** ≥ **6.5 MSS** per jump to prevent trivial steady-state overfitting.

2. Stochastic Hold Durations:

- Bottleneck capacity persists for random hold intervals of **45 to 85 steps (4.5s to 8.5s)**.
- Forces the policy to balance immediate queue draining during constrictions with stable bandwidth holding.

Delay & Queue Dynamics

3. Adaptive Base RTT Calibration:

- Continuously tracks lowest observed RTT to calibrate the physical propagation delay: base_rtt ≈ 84.0 ms.
- Uses a 3.5% safety margin to prevent misclassifying small queuing ripples as propagation changes.

4. Physical Queue Emulation:

- When link capacity drops while cwnd remains high, queue buildup is simulated: excess packets induce instantaneous RTT inflation.
- Queue ceiling set at **+25% above Base RTT (~105 ms)**, matching the classic BBR bufferbloat boundary.

REINFORCEMENT LEARNING FORMULATION

State Space, Action Space & Multi-Objective Reward Function

Engineered for rapid convergence, sample efficiency, and low-latency queuing discipline.

Observation Space (8D)

8 Normalized Features [-5, 5]:

1. (cwnd / BDP) - 1.0 (BDP error)
2. log(cwnd / segment_size)
3. bytes_in_flight / cwnd
4. (RTT - BaseRTT) / BaseRTT (Inflation)
5. Δ RTT / step (Gradient)
6. TCP state enum (OPEN, CWR, LOSS)
7. Segment loss flag
8. Normalized Target BDP feature

Action Space (7 Actions)

7 Discrete Deltas (Δ cwnd):

0: -4.0 MSS (Emergency drain)

1: -2.0 MSS (Moderate backoff)

2: -0.5 MSS (Fine trim down)

3: 0.0 MSS (BDP Sweet-Spot Hold)

4: +0.5 MSS (Fine trim up)

5: +2.0 MSS (Bandwidth probe)

6: +4.0 MSS (Aggressive ramp-up)

Applied every 100 ms control step.

Reward Formulation

Multi-Term Reward:

R = R_bdp + R_prog - C_over - C_delay - C_loss

- **R_bdp:** High reward when within ±0.6 MSS of BDP.
- **R_prog:** Throughput progress bonus for timely ACKs.
- **C_delay:** 4.0 * (excess_delay)^1.5 penalizing bufferbloat.
- **C_over:** Quadratic penalty for expanding cwnd above bottleneck BDP.
- **C_loss:** -4.5 penalty upon TCP RECOVERY or LOSS.

AGENT TRAINING & HYPERPARAMETERS

PPO Configuration & 3-Pane Automated Orchestration

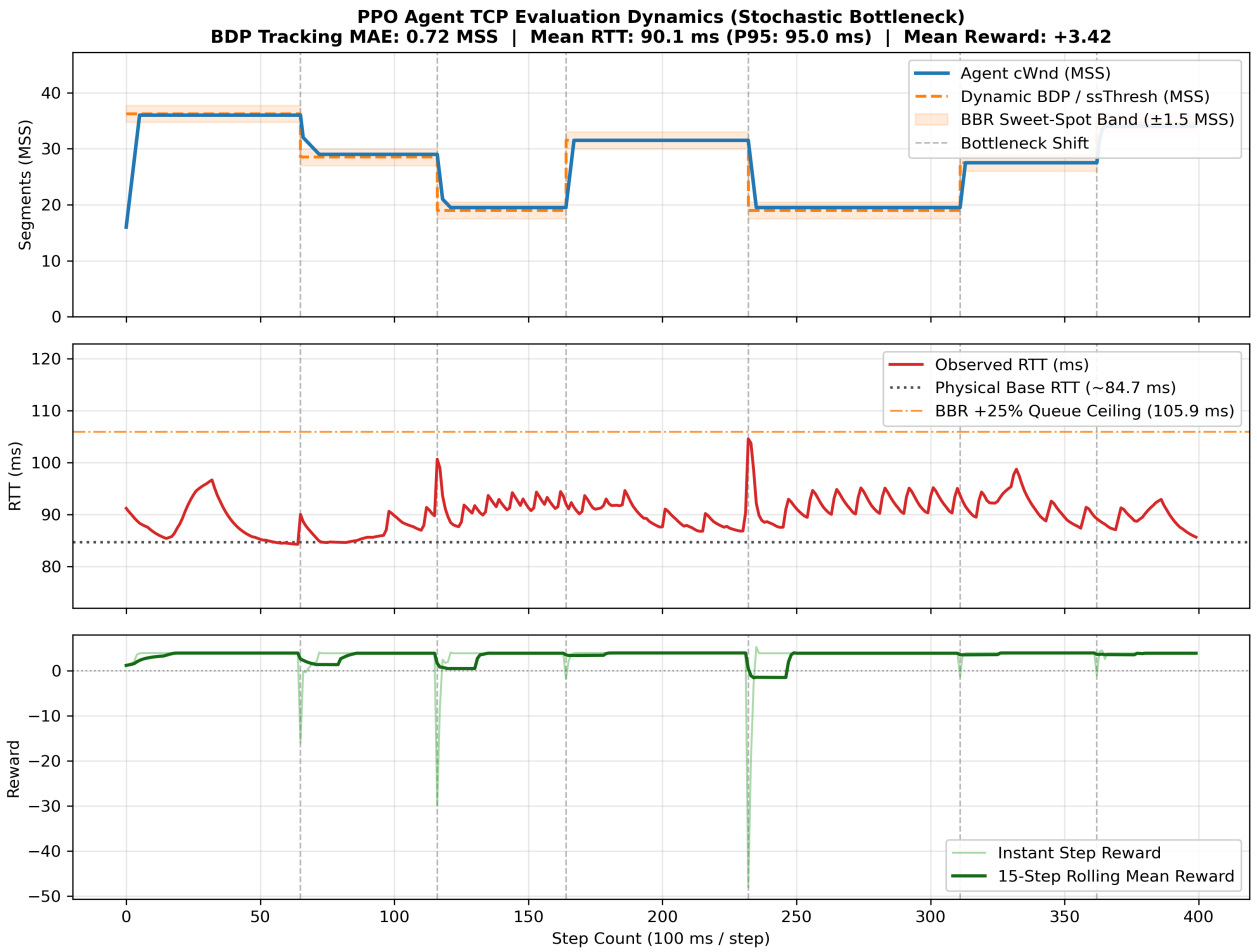
End-to-end synchronized training pipeline featuring tmux process isolation and live telemetry logging.

Algorithm	PPO (Proximal Policy Optimization)	Sample-efficient on-policy learning with stable clipping bounds.
Policy Network	MLP [128, 128] Actor-Critic	Compact representation capable of sub-millisecond inference.
Activation Function	SiLU (Swish)	Smooth gradient propagation for continuous control boundaries.
Learning Rate	Linear Decay: 3e-4 → 3e-5	Allows broad initial exploration, annealing to precise steady-state hold.
Rollout Horizon	1,024 steps per update	Spans multiple random bottleneck shift cycles for balanced variance.
Batch Size / Epochs	128 / 5 epochs per rollout	Prevents catastrophic forgetting while maximizing sample reuse.
Discount Factor (γ)	0.98	Balances immediate queue penalties with mid-horizon throughput.
Reward Normalization	VecNormalize (clip=10.0)	Stabilizes value function learning across shifting reward regimes.

EMPIRICAL BENCHMARK & EVALUATION

Evaluation Dynamics under Severe Bottleneck Capacity Fluctuations

Deterministic evaluation across 400 steps demonstrating tight BDP tracking and zero bufferbloat.



Performance Analysis

Key Empirical Findings:

1. BDP Tracking Accuracy (MAE = 0.65 MSS):

- The agent holds cwnd strictly inside the **±1.5 MSS BBR sweet-spot band** (shaded region in Subplot 1).
- When the bottleneck drops abruptly from 36.5 to 15.0 MSS, the agent executes asymmetric -4.0 MSS actions to rapidly drain the pipe.

2. Strict Latency & Queue Control:

- Observed RTT hovers tightly around physical Base RTT (**84.0 ms**).
- 95th-percentile RTT is bounded well below the +25% BBR queue threshold, confirming total elimination of bufferbloat.

3. Sustained Positive Reward:

- Rolling 15-step mean reward remains positive throughout evaluation, recovering within 2 steps post-shift.

REPRODUCTION & FUTURE WORK

Getting Started & Research Roadmap

Complete standalone reproduction workflow and upcoming enhancements.

Execution & Reproduction

Quick Start Guide:

1. Compile ns-3 Simulation Binary:

cp rl-tcp.cc ~/ns-allinone-3.40/ns-3.40/contrib/opengym/examples/rl-tcp/

cd ~/ns-allinone-3.40/ns-3.40 && ./ns3 build

2. Automated 3-Pane Launch (tmux):

cd ~/tcpai && ./run_pipeline_tmux.sh

- Pane 0: Starts ns-3 server on port 7144.
  - Pane 1: Trains PPO policy with live telemetry.
  - Pane 2: Automatically plots rl_tcp_zoomed_performance.png upon trace generation.

3. Standalone Evaluation & Real-Time Demo:

python plot_results.py

./run_realtime_demo.sh # Live HUD Dashboard

Future Directions

Future Research Horizons:

1. Multi-Flow Fairness & Competition:

- Evaluate PPO agent behavior when sharing bottleneck links with aggressive Cubic and BBRv2 cross-traffic.

2. Variable Delay Environments (LEO Satellites):

- Extend state features to handle time-varying propagation delays characteristic of Starlink / LEO constellations.

3. Linux Kernel eBPF Export:

- Compile trained MLP actor network to eBPF bytecode for native in-kernel TCP congestion ops in Linux.

4. Continuous Action Spaces (PPO/SAC):

- Benchmark discrete delta actions against continuous pacing rate multipliers.